Lecture 30 — Combinatorics, Inclusion-Exclusion, Probability & Expected Value

> Year Curriculum · Prerequisites: Week 9 (bitmasks), Week 11 (modular arithmetic, inverses, nCr mod p), Weeks 14–15 (DP) · Estimated time: 3 hours

1. Overview

Counting problems ask "how many objects satisfy …?" and probability problems ask "how likely is …?" or "what is the average …?". The two are closely linked: under a uniform distribution, a probability is just a count divided by another count, and an expected value is a sum over outcomes divided by their number.

This lecture builds a toolkit in four layers:

1. **Basic counting**: product rule, permutations, combinations, multinomial coefficients, and how to evaluate them modulo a prime.
2. **Classic patterns**: stars and bars, Catalan numbers.
3. **Inclusion-exclusion**: the standard way to count objects that avoid a set of "bad" properties.
4. **Probability and expectation**: linearity of expectation with indicator variables (the single most useful idea here), and DP over probabilities and expectations, including answers printed modulo a prime.

2. Counting fundamentals

2.1 Rules and basic quantities

- **Product rule**: if a choice is made in stages with a_1, a_2, … options at each stage (independent of earlier choices' identities), the total is $a_1 \cdot a_2 \cdot \ldots$
- **Sum rule**: if objects split into disjoint classes, add the class counts.

From these:

Objects	Count
length-k sequences over n symbols	n^k
permutations of n distinct items	n!
ordered selections of k out of n	P(n,k) = n!/(n−k)!

Objects	Count
k-subsets of an n-set	$C(n,k) = n!/(k!(n-k)!)$
distinct arrangements of a multiset with counts $c_1,\ldots,c_m$ ($\Sigma c_i = n$)	$n!/(c_1!\cdot\ldots\cdot c_m!)$

The last one (the **multinomial coefficient**) solves CSES 1715 Creating Strings II: the number of distinct strings formed from the letters of s is |s|! divided by the factorial of each letter's count. For `aabac` that is 5!/(3!·1!·1!) = 20.

Useful identities: $C(n,k) = C(n-1,k-1) + C(n-1,k)$ (Pascal), $C(n,k) = C(n,n-k)$, $\Sigma_k C(n,k) = 2^n$, and Vandermonde $\Sigma_j C(a,j)C(b,k-j) = C(a+b,k)$.

2.2 Evaluating nCr modulo a prime

From Week 11: precompute `fact[i]` and `inv_fact[i]` up to the largest n you need, then `C(n,k) = fact[n]·inv_fact[k]·inv_fact[n-k]`. Compute `inv_fact[N]` with one Fermat inverse and the rest downward via `inv_fact[i-1] = inv_fact[i]·i`. This is O(N) setup and O(1) per query. If the modulus is not prime, or n is tiny, use Pascal's triangle ($O(n^2)$) instead. Always make C return 0 when $k < 0$ or $k > n$; many formulas rely on it.

2.3 Stars and bars

The number of ways to write m as an ordered sum of n **non-negative** integers $x_1 + \ldots + x_n = m$ equals the number of ways to place $n - 1$ bars among m stars:

```
**|*||***   ↔   (2, 1, 0, 3)
```

There are $m + n - 1$ positions, of which $n - 1$ are bars: **$C(m + n - 1, n - 1)$**. CSES 1716 Distributing Apples (n children, m apples) is exactly this; with n = 3, m = 2 the answer is C(4,2) = 6.

Variants: if each $x_i \geq 1$, substitute $y_i = x_i - 1$ to get $C(m - 1, n - 1)$. If each $x_i \leq U$, use inclusion-exclusion over the set of variables that exceed U (Section 3).

2.4 Catalan numbers

$C_n = C(2n, n)/(n + 1) = 1, 1, 2, 5, 14, 42, 132, 429, \ldots$ counts balanced bracket sequences of n pairs, full binary trees with n + 1 leaves, triangulations of a convex (n + 2)-gon, and many more. Recurrence: $C_0 = 1$, $C_{n+1} = \Sigma_i C_i C_{n-i}$ (split at the bracket matching the first one).

Reflection proof. View a bracket sequence as a walk: "(" = +1, ")" = −1. All walks of 2n steps ending at 0: C(2n, n). A *bad* walk touches −1 at some point; reflect the part after the first touch. This bijects bad walks with walks ending at −2, i.e. with $n - 1$ up-steps: C(2n, n + 1). So

$C_n = C(2n,n) - C(2n,n+1)$. The same reflection trick counts paths constrained to stay on one side of any line $y = x + c$, and handles CSES 2187 Bracket Sequences II (a given prefix).

3. Inclusion-exclusion

3.1 The principle

For finite sets $A_1, \ldots, A_k$:

$|A_1 \cup \ldots \cup A_k| = \Sigma$ over non-empty $S \subseteq \{1..k\}$ of $(-1)^{|S|+1} \cdot |\cap_{i \in S} A_i|$.

Equivalently, the number of elements of the universe U lying in **none** of the sets is Σ over all S (including empty) of $(-1)^{|S|} \cdot |\cap_{i \in S} A_i|$, with the empty intersection being U.

Why it is true: an element lying in exactly $t \geq 1$ of the sets is counted $\Sigma_{j=1..t} (-1)^{j+1} C(t,j) = 1$ time. The art is choosing the "bad" properties A_i so that **intersections are easy to count**, even when the union is not.

3.2 Divisibility (subset form)

CSES 2185 Prime Multiples: count the integers in [1, n] ($n \leq 10^{18}$) divisible by at least one of $k \leq 20$ given distinct primes. Let A_i be the multiples of a_i. The intersection over S is the multiples of $\prod_{i \in S} a_i$, of which there are $\lfloor n / \prod \rfloor$. Enumerate the 2^k masks, add for odd $|S|$ and subtract for even. Products can overflow 64 bits; stop multiplying as soon as the product exceeds n (the term is then 0). Cost $O(2^k \cdot k) \approx 2 \cdot 10^7$.

Same idea: counting integers in [1, n] coprime to m — factor m (at most ~15 distinct primes for $m \leq 10^{18}$) and subtract numbers sharing a prime. This leads to the Möbius function, covered in Regional lecture 7.

3.3 Derangements (symmetric form)

When the size of an intersection depends only on $|S| = j$, group the terms: $\Sigma_j (-1)^j C(k, j) \cdot f(j)$. For derangements (permutations without fixed points, CSES 1717), A_i = "i is a fixed point". Fixing a specific set of j points leaves $(n - j)!$ permutations, so

$D(n) = \Sigma_{j=0..n} (-1)^j C(n, j)(n - j)! = n! \cdot \Sigma_{j=0..n} (-1)^j / j!$.

A cleaner computational form is the recurrence **$D(n) = (n - 1)(D(n - 1) + D(n - 2))$**, $D(0) = 1$, $D(1) = 0$. (Element n goes to some position $i \neq n$ in $n - 1$ ways; then either i goes to n — a 2-cycle, $D(n - 2)$ — or it does not, which behaves like a derangement of $n - 1$ elements.) Since $\Sigma(-1)^j/j! \to 1/e$, about 36.8% of permutations are derangements.

3.4 A two-sequence example

AtCoder ABC 172 E (NEQ): count pairs of sequences A, B of length N with values in 1..M, each sequence having pairwise distinct elements, and $A_i \neq B_i$ for every i. Fix A: P(M, N) ways. By symmetry, the count of B is independent of A. For a chosen set of j positions where B must equal A, the remaining N − j positions use M − j unused values: P(M − j, N − j). Thus

answer = P(M, N) · Σ_{j=0..N} $(-1)^j$ C(N, j) P(M − j, N − j),

an O(N) computation with factorial tables. Check: N = M = 2 gives 2 · (2 − 2 + 1) = 2.

3.5 Counting surjections

The number of functions from an n-set onto an m-set (every value used) is Σ_{j=0..m} $(-1)^j$ C(m, j)$(m - j)^n$: subtract functions that miss some chosen set of j values. This is m! · S(n, m), with S the Stirling numbers of the second kind.

4. Probability and expectation

4.1 Basics

For a finite uniform sample space Ω, P(E) = |E|/|Ω|. Rules: P(A ∪ B) = P(A) + P(B) − P(A ∩ B) (inclusion-exclusion again), P(A ∩ B) = P(A)P(B) when A and B are independent, and P(A | B) = P(A ∩ B)/P(B).

Problems either accept a floating-point answer within an absolute/relative error (print with `printf("%.10f\n", x)` or `setprecision(10)`), or ask for $P \cdot Q^{-1}$ mod a prime. In the modular case, represent each probability as a residue — a probability of 30% is `30 · inv(100)` — and never pass through doubles. This is legitimate because the true answer is a rational number whose denominator is coprime with the modulus.

4.2 Expected value and linearity

For a random variable X with finitely many values, $E[X] = \Sigma_x\, x \cdot P(X = x)$. The key fact is:

Linearity of expectation: E[X + Y] = E[X] + E[Y] and E[cX] = cE[X], for *any* random variables, independent or not.

Combined with **indicator variables** — write the quantity as X = Σ I_i where I_i = 1 if "event i happens" — we get E[X] = Σ P(event i). The events can be heavily dependent; we never need the joint distribution.

Examples:

- *Fixed points.* For a uniformly random permutation, $P(\pi(i) = i) = 1/n$, so the expected number of fixed points is $n \cdot 1/n = 1$.
- *Inversions.* Each pair $i < j$ is inverted with probability ½, so $E = n(n-1)/4$.
- *Inversion Probability (CSES 1728).* x_i is uniform on $[1, r_i]$ independently; $E[\#\text{inversions}] = \Sigma_{i<j} P(x_i > x_j)$, each term computed by summing over x_i in $O(r)$. With $n \le 100$ and $r_i \le 100$ this is $\sim 5 \cdot 10^5$ operations.
- *Game on Tree (CF 280C).* A node is removed by its own pick iff it is chosen before all of its ancestors. Among the $d = \text{depth}(v)$ nodes on the root path, by symmetry each is equally likely to be chosen first, so $P = 1/d$ and $E[\#\text{steps}] = \Sigma_v 1/\text{depth}(v)$.

4.3 Geometric waiting times; coupon collector

If each trial succeeds independently with probability p, the expected number of trials until the first success is $1/p$ (from $E = 1 + (1-p)E$). The **coupon collector** problem — draw uniformly from n types until all are seen — splits into phases: having i types, the next new type arrives with probability $(n-i)/n$, so phase i takes $n/(n-i)$ draws in expectation. Linearity gives $n \cdot (1 + 1/2 + \ldots + 1/n) = n \cdot H_n \approx n \ln n$.

4.4 Distribution of a maximum

CSES 1727 Candy Lottery / CF 453A: n independent uniform values on 1..k; find E[max]. The CDF is easy: $P(\max \le x) = (x/k)^n$, so $P(\max = x) = (x/k)^n - ((x-1)/k)^n$ and $E[\max] = \Sigma_x x \cdot P(\max = x)$. Equivalently, the tail-sum formula for non-negative integer variables, $E[X] = \Sigma_{x \ge 1} P(X \ge x)$, gives $E[\max] = \Sigma_{x=1..k} (1 - ((x-1)/k)^n)$. Use `pow` in doubles: computing x/k first keeps numbers in [0, 1]; never compute x^n directly.

4.5 Probability DP (forward)

When the process has a natural state, push **probability mass** forward. CSES 1725 Dice Probability: `dp[t][s]` is the probability that after t throws the sum is s; `dp[t+1][s+f] += dp[t][s]/6`. The answer is $\Sigma_{s=a..b} dp[n][s]$. AtCoder DP I (Coins) is the same pattern with biased coins: `dp[i][h]` = probability of h heads among the first i coins.

4.6 Expected-value DP (backward)

For expectations, define E[state] = expected remaining cost from state and compute **backwards** from terminal states: $E[s] = \text{cost}(s) + \Sigma P(s \to s') E[s']$. If a state can transition to itself (a self-loop with probability p), move that term to the left: $E = c + pE + \Sigma\ldots \Longrightarrow E = (c + \Sigma\ldots)/(1-p)$. AtCoder DP J (Sushi) is the classic example: the state is (#plates with 1, 2, 3 pieces), and choosing an empty plate is the self-loop.

CF 1265E Beautiful Mirrors. Mirror i answers "yes" with probability p_i; yes moves on (done after mirror n), no restarts from mirror 1. Let E_i be the expected days starting at mirror i, with E_{n+1} = 0: $E_i = 1 + p_i E_{i+1} + (1 - p_i)E_1$. Expanding $E_1 = 1 + p_1(1 + p_2(\ldots)) + (1 - \ldots)E_1$ and solving for E_1 yields

$E_1 = (1 + p_1 + p_1p_2 + \ldots + p_1 \cdots p_{n-1}) / (p_1 \cdots p_n)$.

With p = (10%, 20%, 50%): (1 + 0.1 + 0.02)/0.01 = 112. The modular version only needs prefix products and one inverse.

5. Reference implementations

5.1 Combinatorics toolkit: stars and bars, Catalan, derangements

Reads n and m, prints the number of ways to distribute m apples among n children (CSES 1716), plus a few small Catalan and derangement values as a self-check.

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1'000'000'007;
const int MAXN = 2'000'001;
vector<long long> fact(MAXN), inv_fact(MAXN);

long long power(long long b, long long e) {
    long long r = 1;
    b %= MOD;
    for (; e > 0; e >>= 1, b = b * b % MOD)
        if (e & 1) r = r * b % MOD;
    return r;
}

void init() {
    fact[0] = 1;
    for (int i = 1; i < MAXN; i++) fact[i] = fact[i - 1] * i % MOD;
    inv_fact[MAXN - 1] = power(fact[MAXN - 1], MOD - 2);
    for (int i = MAXN - 1; i > 0; i--) inv_fact[i - 1] = inv_fact[i] * i % MOD;
}

long long C(int n, int k) {
    if (k < 0 || k > n || n < 0) return 0;
    return fact[n] * inv_fact[k] % MOD * inv_fact[n - k] % MOD;
}

long long catalan(int n) { return C(2 * n, n) * power(n + 1, MOD - 2) % MOD; }

vector<long long> derangements(int n) {
    vector<long long> d(n + 1, 0);
    d[0] = 1;
    if (n >= 1) d[1] = 0;
```

```
    for (int i = 2; i <= n; i++) d[i] = (i - 1) * ((d[i - 1] + d[i - 2]) % MOD) % MOD;
    return d;
}

int main() {
    init();
    int n, m;
    cin >> n >> m;
    cout << C(n + m - 1, n - 1) << "\n";      // stars and bars
    for (int i = 0; i <= 6; i++) cerr << catalan(i) << " ";   // 1 1 2 5 14 42 132
    cerr << "\n";
    for (long long x : derangements(6)) cerr << x << " ";     // 1 0 1 2 9 44 265
    cerr << "\n";
}
```

5.2 Inclusion-exclusion — CSES 2185 Prime Multiples

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    long long n;
    int k;
    cin >> n >> k;
    vector<long long> a(k);
    for (auto& x : a) cin >> x;
    long long ans = 0;
    for (int mask = 1; mask < (1 << k); mask++) {
        long long prod = 1;
        bool over = false;
        for (int i = 0; i < k && !over; i++) {
            if (!(mask >> i & 1)) continue;
            if (prod > n / a[i]) over = true;
            else prod *= a[i];
        }
        if (over) continue;
        long long cnt = n / prod;
        if (__builtin_popcount(mask) & 1) ans += cnt;
        else ans -= cnt;
    }
    cout << ans << "\n";
}
```

Sample: `20 2` / `2 5` → 12 (2, 4, 5, 6, 8, 10, 12, 14, 15, 16, 18, 20).

5.3 Expected maximum — CSES 1727 Candy Lottery

```
#include <bits/stdc++.h>
using namespace std;

int main() {
```

```
    int n, k;
    cin >> n >> k;
    double ans = 0;
    for (int x = 1; x <= k; x++) {
        double pLe = pow((double)x / k, n);         // P(max <= x)
        double pLt = pow((double)(x - 1) / k, n);   // P(max <= x-1)
        ans += x * (pLe - pLt);
    }
    printf("%.6f\n", ans);
}
```

Sample: `2 3` → 2.444444 (= 22/9).

5.4 Linearity on a tree — CF 280C Game on Tree

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<vector<int>> g(n + 1);
    for (int i = 0; i < n - 1; i++) {
        int a, b;
        cin >> a >> b;
        g[a].push_back(b);
        g[b].push_back(a);
    }
    vector<int> depth(n + 1, 0);
    depth[1] = 1;
    vector<int> st = {1};                 // iterative DFS: n can be 1e5
    double ans = 0;
    while (!st.empty()) {
        int v = st.back(); st.pop_back();
        ans += 1.0 / depth[v];
        for (int u : g[v]) if (!depth[u]) { depth[u] = depth[v] + 1; st.push_back(u); }
    }
    printf("%.10f\n", ans);
}
```

Sample: a path of 2 nodes → 1.5; a star with center 1 and 2 leaves → 2.0.

5.5 Modular expected value — CF 1265E Beautiful Mirrors

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 998244353;
```

```cpp
long long power(long long b, long long e) {
    long long r = 1;
    b %= MOD;
    for (; e > 0; e >>= 1, b = b * b % MOD)
        if (e & 1) r = r * b % MOD;
    return r;
}

int main() {
    int n;
    cin >> n;
    long long inv100 = power(100, MOD - 2);
    long long numer = 0, prefix = 1;      // prefix = p1 * ... * p_i
    for (int i = 0; i < n; i++) {
        long long p;
        cin >> p;
        numer = (numer + prefix) % MOD;   // adds 1, p1, p1p2, ..., p1..p_{n-1}
        prefix = prefix * (p * inv100 % MOD) % MOD;
    }
    cout << numer * power(prefix, MOD - 2) % MOD << "\n";
}
```

Samples: `1` / `50` → 2; `3` / `10 20 50` → 112.

6. Common mistakes

1. **C(n,k) without bounds checks.** Formulas like $\Sigma(-1)^j$ C(N, j)·… often hit k > n; return 0.
2. **Factorial table too small.** Stars and bars uses n + m − 1; Catalan uses 2n. Size the table for the largest argument.
3. **Overflow in inclusion-exclusion products.** Check `prod > n / a[i]` before multiplying, or use `__int128`.
4. **Sign conventions.** For the *union*, odd |S| adds; for the *complement* ("in none"), even |S| adds (including the empty set = everything). Mixing them gives answers off by exactly the total.
5. **Negative residues.** After subtraction do `(x % MOD + MOD) % MOD`.
6. **Confusing linearity with independence.** E[X + Y] = E[X] + E[Y] always; E[XY] = E[X]E[Y] only for independent variables. Similarly, P(A ∩ B) = P(A)P(B) needs independence.
7. **Doubles in a modular problem.** Once you convert a probability to `double` there is no way back to the exact residue.
8. **Forward vs. backward DP.** Probabilities of reaching states flow forward from the start; expectations of remaining cost flow backward from the end. Mixing the two directions usually double counts.
9. **Output precision.** Print at least 6–10 digits after the decimal point, as the statement demands.
10. **Deep recursion** on trees with 10^5 nodes (path-shaped). Use an explicit stack as in 5.4.

7. Practice problems

Counting

- CSES 1079 — Binomial Coefficients (https://cses.fi/problemset/task/1079)
- CSES 1715 — Creating Strings II (https://cses.fi/problemset/task/1715)
- CSES 1716 — Distributing Apples (https://cses.fi/problemset/task/1716)
- CSES 2064 — Bracket Sequences I (https://cses.fi/problemset/task/2064)
- CSES 2187 — Bracket Sequences II (https://cses.fi/problemset/task/2187)

Inclusion-exclusion

- CSES 1717 — Christmas Party (https://cses.fi/problemset/task/1717)
- CSES 2185 — Prime Multiples (https://cses.fi/problemset/task/2185)
- AtCoder ABC 172 E — NEQ (https://atcoder.jp/contests/abc172/tasks/abc172_e)
- CSES 2417 — Counting Coprime Pairs (https://cses.fi/problemset/task/2417)

Probability and expectation

- CSES 1725 — Dice Probability (https://cses.fi/problemset/task/1725)
- CSES 1726 — Moving Robots (https://cses.fi/problemset/task/1726)
- CSES 1727 — Candy Lottery (https://cses.fi/problemset/task/1727)
- CSES 1728 — Inversion Probability (https://cses.fi/problemset/task/1728)
- Codeforces 453A — Little Pony and Expected Maximum (https://codeforces.com/problemset/problem/453/A)
- Codeforces 280C — Game on Tree (https://codeforces.com/problemset/problem/280/C)
- Codeforces 1265E — Beautiful Mirrors (https://codeforces.com/problemset/problem/1265/E)
- Codeforces 518D — Ilya and Escalator (https://codeforces.com/problemset/problem/518/D)
- AtCoder Educational DP Contest I — Coins (https://atcoder.jp/contests/dp/tasks/dp_i)
- AtCoder Educational DP Contest J — Sushi (https://atcoder.jp/contests/dp/tasks/dp_j)

8. Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, chapters 22 (combinatorics) and 24 (probability)
- cp-algorithms: "Inclusion-Exclusion Principle" (https://cp-algorithms.com/combinatorics/inclusion-exclusion.html), "Catalan Numbers", "Stars and bars"
- R. Graham, D. Knuth, O. Patashnik, *Concrete Mathematics*, chapters 5 and 8
- Codeforces blog: "Linearity of expectation" tutorials and the CF 280C editorial
- Next steps: Möbius inversion (Regional 7), Burnside/Pólya and generating functions (World Finals 7)